

SMART-PLANT: SISTEMA IOT DE MONITOREO Y RIEGO AUTOMÁTICO DE PLANTAS EN ENTORNOS DOMÉSTICOS

Axel Valenzuela¹, Gabriela Bernard²

Resumen

Palabras clave:

Abstract

Keywords:

1. Introducción

El avance del Internet de las Cosas (IoT) ha transformado la forma en que los sistemas embebidos interactúan con el entorno físico, permitiendo la recolección automatizada de datos y la ejecución de acciones en respuesta a condiciones detectadas por sensores. En el campo de la agricultura de precisión y el cuidado de plantas, estas tecnologías ofrecen una oportunidad para optimizar el uso del agua y mejorar las condiciones de crecimiento mediante sistemas inteligentes capaces de operar con mínima intervención humana.

En este contexto, el presente trabajo describe el desarrollo de Smart-Plant, un sistema de monitoreo y riego automático de plantas basado en IoT, cuyo objetivo es detectar en tiempo real la condición de humedad del suelo y activar el riego de manera autónoma cuando sea necesario. El sistema integra hardware de bajo costo y consumo (módulos ESP32) con tecnologías de comunicación estándar en IoT (protocolo MQTT) y una arquitectura de software escalable (Node.js, MongoDB), accesible a través de un dashboard web.

1.1 Objetivos

Como objetivo general, se propone: diseñar e implementar un sistema embebido IoT modular que integre sensores de temperatura y humedad para el monitoreo ambiental y riego automatizado de plantas en espacios domésticos y de oficina, gestionado a través de internet mediante una interfaz de usuario interactiva.

Se definen, además, una serie de objetivos específicos que buscan ser cumplidos durante el desarrollo del proyecto:

1. Acondicionar y calibrar sensores de humedad y sensores de temperatura para la adquisición de datos del suelo en tiempo real.
2. Programar el firmware del microcontrolador para interpretar los datos de los sensores y accionar las bombas de agua de manera autónoma cuando la humedad caiga por debajo de los umbrales predefinidos.
3. Establecer una infraestructura de comunicación vía WiFi basada en el protocolo MQTT para transmitir la telemetría a un servidor y recibir comandos de actuación.
4. Crear una aplicación/dashboard que permita visualizar el histórico de riegos, el estado actual de las plantas, y que cuente con un control para activar el riego manualmente.
5. Configurar el firmware y el servidor para que el sistema sea modular, permitiendo el emparejamiento y la gestión de múltiples nodos de monitoreo en la misma red.

Commented [AV1]: Agregar justificación

1.2 Estado del arte en sistemas IoT de monitoreo y riego automático

El monitoreo y riego automatizado mediante IoT ha sido abordado desde múltiples perspectivas en la literatura reciente, con énfasis en tres dimensiones principales: la plataforma de hardware empleada, el protocolo de comunicación adoptado y la estrategia de control utilizada. A continuación, se resumen los trabajos encontrados considerados más relevantes para el proyecto.

1.2.1 Plataformas de hardware predominantes

En el ámbito de IoT doméstico y agrícola, los microcontroladores más citados en trabajos recientes son los basados en la familia ESP-32 y, en menor medida, variantes de Arduino con módulo Wi-Fi (p. ej., ESP8266 como núcleo de red). El ESP32-WROOM-32U se ha consolidado como plataforma de referencia en sistemas de riego con monitoreo IoT, al integrar procesamiento, Wi-Fi 802.11 y Bluetooth en un solo módulo, con memoria RAM suficiente y múltiples interfaces (SPI, I2C, UART) para la conexión de sensores y actuadores.

1.2.2 Protocolos de comunicación y herramientas de software

Para el transporte de datos desde los nodos sensores hasta un servidor o broker local, el protocolo MQTT (con implementaciones como Mosquitto o HiveMQ) aparece de manera consistente en la literatura como la opción más adecuada para IoT doméstico, dado su diseño ligero, orientado a eventos y con bajo overhead. Su modelo de publicación/suscripción lo hace especialmente apto para la transmisión de telemetría de sensores de humedad, temperatura y estado de actuadores.

1.2.3 Sistemas de riego automatizado con umbrales fijos

Diversos autores han implementado sistemas de riego automatizado para huertos caseros usando ESP32 como nodo sensor y Raspberry Pi como broker MQTT y servidor de reglas (Hernández González et al., 2025; Acosta Gallo y Cuaical Angulo, 2021; Perea Alvarado y Bustamante Briones, 2024). Otros trabajos extienden este enfoque a invernaderos y cultivos protegidos, incorporando sensores de humedad del suelo (FC-28) y decisiones autónomas basadas en umbrales de humedad y climatología (Angulo Martínez, 2025; Gómez Zea et al., 2025). En el ámbito del código abierto, repositorios como RiegoJardin-ESP32 y Smart Farming ESP32 —aunque sin revisión académica formal— constituyen referencias técnicas relevantes que implementan lecturas de sensores DHT11 y FC-28, accionamiento de relés y visualización mediante NodeRED (Richard-Roman, 2026; Lily-osp, 2025). Estos antecedentes demuestran la viabilidad técnica del enfoque y han permitido cuantificar sus beneficios: Hasib y Ahsanul Sarkar Akib (2026), en una evaluación con plantas de interior y macetas, lograron una precisión del 92 % en la humedad óptima del suelo y una reducción del 40 % en el consumo de agua respecto al riego manual tradicional.

1.2.4 Sistemas predictivos e inteligencia artificial

Más allá del control por umbrales estáticos, trabajos recientes proponen estrategias predictivas que incorporan aprendizaje automático. Ruiz (2025) desarrolla un sistema

Commented [AV2]: Cambiar a Antecedentes en lugar de Estado del Arte.

Estado del arte suele ser más amplio y más documentado, cosa para lo cual da poco tiempo.

híbrido en dos capas: un Arduino Nano recolecta datos de humedad capacitiva, temperatura y humedad ambiental y los envía inalámbricamente a una Raspberry Pi 4 local, donde un modelo LSTM cruza la telemetría de las macetas con el pronóstico climático de Open-Meteo para anticipar el riego antes de que la planta alcance estrés hídrico. En una línea complementaria, Oppong (2025) propone migrar los modelos de decisión directamente al microcontrolador, eliminando la dependencia de la nube y reduciendo la latencia y la vulnerabilidad ante cortes de conectividad.

1.2.5 Brechas identificadas

A pesar de los avances documentados, la revisión identifica que la mayoría de los sistemas están orientados a cultivos agrícolas o invernaderos de mediana y gran escala. Se identifica una ausencia de implementaciones en entornos domésticos que combinen escalabilidad modular para la gestión simultánea de múltiples nodos de monitoreo con una interfaz de usuario interactiva que permita visualización histórica y control remoto. Estas brechas se corresponden con los objetivos del presente proyecto: la arquitectura modular propuesta y el dashboard de control buscan aportar novedad y utilidad al sistema propuesto.

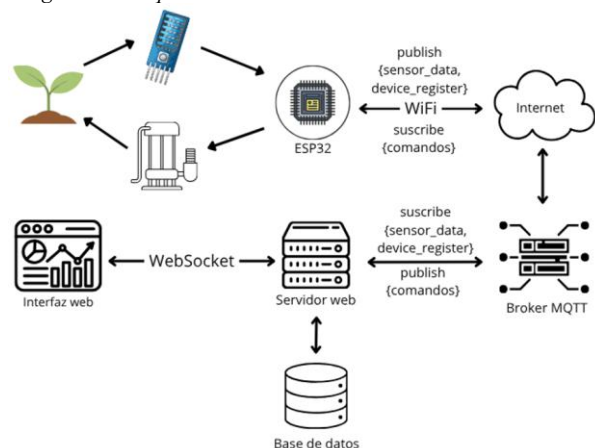
2. Arquitectura del sistema

La arquitectura del sistema Smart-Plant ha sido diseñada bajo un paradigma de IoT distribuido, separando las responsabilidades en capas físicas, de comunicaciones y de aplicación.

En la Figura 1 se ilustra el diagrama general de la arquitectura del sistema.

Figura 1.

Diagrama de arquitectura del sistema.



Commented [AV3]: Agregar subtítulos a los elementos de la capa de recepción

La arquitectura se divide de la siguiente manera:

1. **Capa de recepción (Hardware):** Compuesta por un módulo central basado en el microcontrolador ESP32, el cual interactúa directamente con el entorno a través de los sensores del suelo y actúa mediante una bomba de agua.

2. **Capa de red (Comunicaciones):** Utiliza el protocolo WiFi para conectar al ESP32 a internet. La transferencia de datos de telemetría y recepción de comandos de riego se gestiona mediante el protocolo MQTT.
3. **Capa de aplicación y datos (Backend/Frontend):** Un servidor desarrollado en NodeJS organiza la lógica de intercambio, procesando los mensajes MQTT a través del broker EMQX, y almacenando los registros históricos y eventos de riego en una base de datos no relacional MongoDB. Finalmente, un dashboard web permite al usuario visualizar el estado y enviar comandos.

3. Tecnologías seleccionadas

3.1 Hardware seleccionado

3.1.1 Microcontrolador

Se seleccionó el ESP32 como microcontrolador principal debido a que integra conectividad Wi-Fi, característica fundamental para el proyecto, ya que permite enviar los datos de los sensores hacia una base de datos en tiempo real y recibir instrucciones desde la interfaz del sistema. Gracias a esta conectividad, el usuario puede monitorear el estado de la planta y acceder a la información desde la plataforma de manera remota.

Además, el ESP32 ofrece buena capacidad de procesamiento, múltiples pines de entrada y salida y bajo consumo energético, lo que permite conectar varios sensores y actuadores al mismo tiempo sin afectar el rendimiento. También resulta conveniente por su facilidad de programación y por la amplia disponibilidad de documentación y soporte para su implementación.

3.1.2 Sensor NPK + temperatura + humedad + conductividad eléctrica

Se seleccionó un sensor multiparámetro capaz de medir nitrógeno, fósforo y potasio (NPK), además de humedad, temperatura y conductividad eléctrica del suelo. Estas variables son esenciales para evaluar el estado general de la planta, ya que permiten conocer tanto la disponibilidad de nutrientes como las condiciones ambientales y del sustrato donde se encuentra sembrada.

La combinación de estas mediciones permite generar un monitoreo más preciso y útil para el usuario, ya que ayuda a detectar necesidades de riego y evaluar si la planta cuenta con condiciones adecuadas para su crecimiento. La conductividad eléctrica complementa el análisis al ofrecer información sobre la concentración de sales y nutrientes presentes en el suelo.

3.1.3 Actuador (relé + bomba peristáltica)

Para el sistema de riego automatizado se seleccionó un relé junto con una bomba peristáltica. El relé permite que el ESP32 controle el encendido y apagado de la bomba de manera segura, ya que actúa como un interruptor entre el microcontrolador y el sistema de alimentación de la bomba, protegiendo el circuito y facilitando la automatización.

La bomba peristáltica fue elegida porque permite suministrar cantidades controladas de agua con mayor precisión, lo que resulta ideal para el cuidado de plantas. El objetivo del sistema es que, cuando la planta necesite riego, el usuario tenga la opción de activar la bomba y que esta se apague automáticamente una vez que se haya suministrado la cantidad de agua necesaria, determinada según la información almacenada en la base de datos para cada tipo de planta.

Commented [AV4]: Agregar justificación de mediciones de nutrientes

3.2 Stack de Software seleccionado

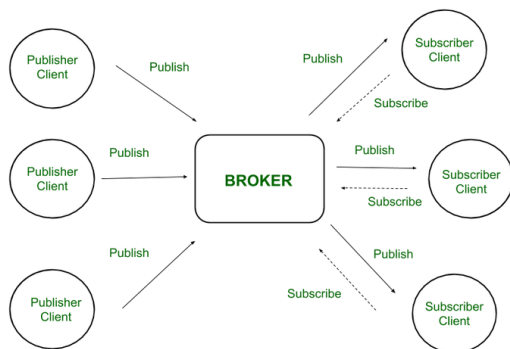
3.2.1 Protocolo y broker MQTT

Para la comunicación entre los nodos ESP32 y el servidor, se adoptó el protocolo MQTT por su bajo overhead y su arquitectura de pub/sub, ideal para dispositivos con recursos limitados. Como broker MQTT se seleccionó EMQX, que ofrece alta disponibilidad, soporte nativo para clústeres, panel de administración integrado y soporte para múltiples protocolos (MQTT, WebSocket, HTTP). EMQX fue evaluado frente a Mosquitto y HiveMQ, siendo seleccionado por su interfaz de administración y capacidades de monitoreo en tiempo real.

MQTT es un protocolo de mensajería simple y liviano que se utiliza para establecer comunicación entre múltiples dispositivos. Es un protocolo basado en TCP que se basa en el modelo de publicación-suscripción. Este protocolo de comunicación es adecuado para transmitir datos entre dispositivos con recursos limitados que tienen un ancho de banda bajo y requisitos de energía bajos. Por lo tanto, este protocolo de mensajería se usa ampliamente para la comunicación en IoT Framework (GeeksforGeeks, 2020).

Figura 2.

Esquema general modelo publicación-suscripción MQTT



El modelo MQTT de publicación-suscripción es un patrón de mensajería que involucra clientes (dispositivos que establecen una conexión con el broker) que envían o reciben mensajes sobre topics (Prokosch, 2025). Los clientes no necesitan estar al tanto de que otros clientes envían o reciben mensajes. La comunicación MQTT involucra al menos a tres partes:

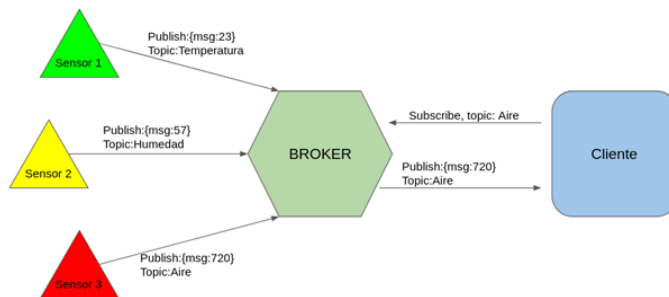
- **Publicadores:** Dispositivos o aplicaciones que envían mensajes que contienen datos sobre topics específicos sin el conocimiento de los destinatarios (suscriptores).
- **Suscriptores:** Dispositivos o aplicaciones que expresan interés en recibir datos publicados sobre topics específicos “suscribiéndose” a estos topics.
- **MQTT Broker:** La pieza central que recibe mensajes de los publicadores y los entrega a los suscriptores en función de sus respectivas suscripciones.

La Figura 3 muestra un diagrama que ilustra este proceso en funcionamiento de manera general.

Figura 3.

Diagrama de ejemplo de funcionamiento MQTT

Commented [AV5]: Buscar cómo citar la imagen con su fuente original



El protocolo opera sobre TCP/IP con un overhead de cabecera mínimo (2 bytes fijos), lo que lo hace eficiente para dispositivos con recursos limitados. MQTT posee además un mecanismo para asegurar que los mensajes son recibidos, y es el QoS (calidad de servicio). Soporta tres niveles de calidad de servicio: at most once (QoS 0), at least once (QoS 1) y exactly once (QoS 2).

3.2.2 Servidor Backend

El servidor de aplicaciones fue implementado en Node.js, un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Google, con un modelo de entrada/salida no bloqueante basado en un bucle de eventos (event loop). Esta arquitectura le permite gestionar de forma eficiente múltiples conexiones concurrentes, entre las que se cuentan el cliente MQTT, los WebSocket abiertos con los navegadores y las operaciones de lectura/escritura en la base de datos, sin recurrir al modelo de hilos del sistema operativo, resultando en un bajo consumo de memoria por conexión.

Se emplearon los paquetes `mqtt.js` para la integración con el broker EMQX, `mongoose` para el modelado de datos en MongoDB, y `express.js` para la exposición de la API REST consumida por el frontend.

Para la comunicación bidireccional entre el servidor y el dashboard web se implementó un servidor WebSocket utilizando la biblioteca `ws`. El protocolo WebSocket establece un canal de comunicación full-duplex y persistente sobre una única conexión TCP, permitiendo al servidor enviar datos al navegador de forma inmediata (server push) sin que el cliente realice pooling (GeeksforGeeks, 2019). Esto resulta esencial para un sistema IoT donde los datos de los sensores se generan continuamente y deben reflejarse en el dashboard con la mínima latencia posible.

Cuando llega un mensaje MQTT, el servidor lo retransmite en broadcast a todos los clientes WebSocket activos; en sentido inverso, los comandos enviados desde el dashboard viajan al servidor vía WebSocket y son enrutados al tópico MQTT del dispositivo correspondiente.

3.2.3 Base de datos

Como sistema de gestión de base de datos se seleccionó MongoDB en su modalidad de servicio en la nube MongoDB Atlas, una base de datos NoSQL orientada a documentos que almacena registros en formato BSON (MongoDB, s.f.). El modelo de datos orientado a documentos se adapta de manera natural a las lecturas de sensores IoT, donde la estructura de cada lectura puede variar entre dispositivos o a lo largo del tiempo del proyecto (por ejemplo, al

agregar nuevos tipos de sensores), sin requerir migraciones de esquema complejas.

Para el modelado de datos y la interacción con MongoDB desde Node.js se utilizó Mongoose, un ODM (Object Document Mapper) que permite definir esquemas tipados con validación automática, índices y métodos de consulta encadenables. La definición de esquemas explícitos en Mongoose, a pesar de la flexibilidad inherente de MongoDB, aporta consistencia de datos y documentación implícita de la estructura esperada para cada colección.

4. Avances del proyecto

4.1 Infraestructura de red

Se estableció con éxito la comunicación bidireccional entre el microcontrolador ESP32 y el servidor centralizado. Para la capa de red, el ESP32 se conecta a internet por medio de WiFi usando la librería `WiFiClientSecure` y establece una conexión segura (TLS/SSL a través del puerto 8883) con un broker MQTT alojado en la nube en la plataforma EMQX.

El firmware ejecuta la conexión en la función `setup()`, mediante la llamada a `WiFi.begin(SSID, PASSWORD)`, seguida de un ciclo de espera que verifica el estado `WiFi.status() != WL_CONNECTED` con un intervalo de 500 ms entre verificaciones.

4.2 Integración con el broker MQTT

Se estableció la comunicación MQTT entre el módulo ESP32 y el broker EMQX Cloud utilizando una conexión cifrada TLS/SSL sobre el puerto 8883. En el firmware, el cliente MQTT se construye sobre una instancia `WiFiClientSecure` y la biblioteca `PubSubClient`, que implementa el protocolo MQTT.

La gestión del ciclo de vida de la conexión MQTT se implementa en la función `connectMQTT()`, que ejecuta un bucle de reconexión mientras el cliente no esté conectado. Al establecer la sesión exitosamente, el cliente se resuscribe a sus tópicos de control y publica el mensaje de registro del dispositivo:

```
void connectMQTT() {
    while (!mqtt.connected()) {
        Serial.print("Conectando al broker MQTT...");

        if (mqtt.connect((deviceID+"-"+PLANT_TYPE).c_str(), MQTT_USERNAME,
MQTT_PASSWORD)) {
            Serial.println("conectado!");

            mqtt.subscribe(topicControl.c_str());
            mqtt.subscribe("control/led/global");

            String reg = "{\"device_id\":\"" + deviceID +
                "\",\"plant_type\":\"" + String(PLANT_TYPE) +
                "\",\"status\":\"online\"}";

            mqtt.publish(topicRegister.c_str(), reg.c_str());
        } else {
            Serial.print(mqtt.state());
        }
    }
}
```

```

        delay(2000);
    }
}
}

```

En el `loop()` principal, si se detecta que el cliente MQTT se ha desconectado, se llama a la función `connectMQTT()` antes de continuar con el ciclo de publicación. La frecuencia de publicación de datos de sensado es de un mensaje cada 2 segundos, gestionada mediante una comparación de `millis()` que evita el uso de `delay()` bloqueante:

```

if (millis() - lastPublish >= 2000) {
    lastPublish = millis();
    int data = random(10,40);

    // Arma el payload en formato JSON
    String payload = "{\"valor\":" + String(data) +
        "\",\"mensaje\":\"Hola desde ESP32\"}";

    mqtt.publish(topicData.c_str(), payload.c_str());
}

```

4.3 Identificación única y registro dispositivos

Cada módulo ESP32 utiliza su dirección MAC como identificador único del dispositivo. Este identificador es globalmente único por diseño de hardware y está disponible en tiempo de ejecución a través de `WiFi.macAddress()` una vez que la conexión de red está activa. El `device_id` cumple una triple función en el sistema: (1) constituye parte del *Client ID* del cliente MQTT, garantizando que múltiples módulos puedan conectarse simultáneamente al broker sin conflictos de identificación; (2) es el sufijo del tópico de datos (`sensor/data/{device_id}`), permitiendo al servidor identificar el origen de cada mensaje sin incluir el ID en el payload; y (3) es el campo de referencia en los documentos de MongoDB que vincula cada lectura y evento de riego con su dispositivo.

Al establecer la conexión MQTT, cada dispositivo publica automáticamente un mensaje de registro en el tópico `device/register` indicando su ID, tipo de planta y estado.

El tipo de planta se configura actualmente como una constante en el firmware. En versiones futuras se espera que este parámetro sea configurable desde el dashboard sin requerir una nueva compilación del firmware.

4.4 Servidor Backend

El servidor backend, desarrollado en NodeJS y utilizando el framework Express, funciona como un cliente MQTT de alto nivel. Este servidor está suscrito a los tópicos de control y telemetría (`sensor/data/#`, `device/register` y `control/led/#`), actuando como puente entre el hardware y la interfaz de usuario. Actualmente, para validar la estabilidad del enlace, el ESP32 publica datos de telemetría simulados (valores aleatorios entre 10 y 40) con una frecuencia de 2 segundos. De manera inversa, el dispositivo está suscrito a su tópico de control específico (`control/led/{deviceId}`), lo que le permite recibir comandos desde el servidor para accionar el actuador simulado (LED integrado), encendiéndolo o apagándolo con una latencia mínima.

La lógica de procesamiento de mensajes MQTT se implementa en el manejador

mqttClient.on('message', ...), que recibe todos los mensajes de los tópicos suscritos y los enruta según el tópico:

- **device/register**: registra o actualiza el dispositivo en el mapa en memoria y emite un evento `device_update` a todos los clientes WebSocket.
- **sensor/data/{device_id}**: extrae el `device_id` del tópico, crea un nuevo documento `SensorReading` en MongoDB con el valor del sensor y el tipo de planta asociado, y emite los datos a los clientes WebSocket en tiempo real.

En sentido contrario, cuando el servidor WebSocket recibe un mensaje desde el navegador, lo parsea como JSON y publica el comando en el tópico MQTT `control/led/{device_id}`. Si el comando es `LED_ON`, además crea un documento `WateringEvent` en MongoDB indicando regado manual y el tipo de planta del dispositivo:

```
ws.on('message', async (message) => {
  const data = JSON.parse(message.toString());
  const deviceID = data.device_id;
  const command = data.command;

  if (deviceID && command) {
    const topic = `control/led/${deviceID}`;
    mqttClient.publish(topic, command);

    if (command === 'LED_ON') {
      await WateringEvent.create({
        device_id: deviceID,
        plant_type: devices.get(deviceID)?.plant_type,
        triggered_by: 'manual'
      });
    }
  }
});
```

El servidor WebSocket, instanciado sobre el servidor HTTP de Express, mantiene conexiones persistentes con todos los navegadores que tienen el dashboard abierto. Al recibir un mensaje MQTT (datos de sensor o actualización de dispositivos), el servidor invoca `broadcastToClients()` que itera los clientes y serializa el objeto de respuesta como JSON:

```
function broadcastToClients(data) {
  wss.clients.forEach(client => {
    if (client.readyState === WebSocket.OPEN)
      client.send(JSON.stringify(data));
  });
}
```

4.5 Base de datos

Para la persistencia de datos, se configuró una conexión exitosa con una base de datos no relacional MongoDB utilizando la librería Mongoose. Se diseñaron dos esquemas o

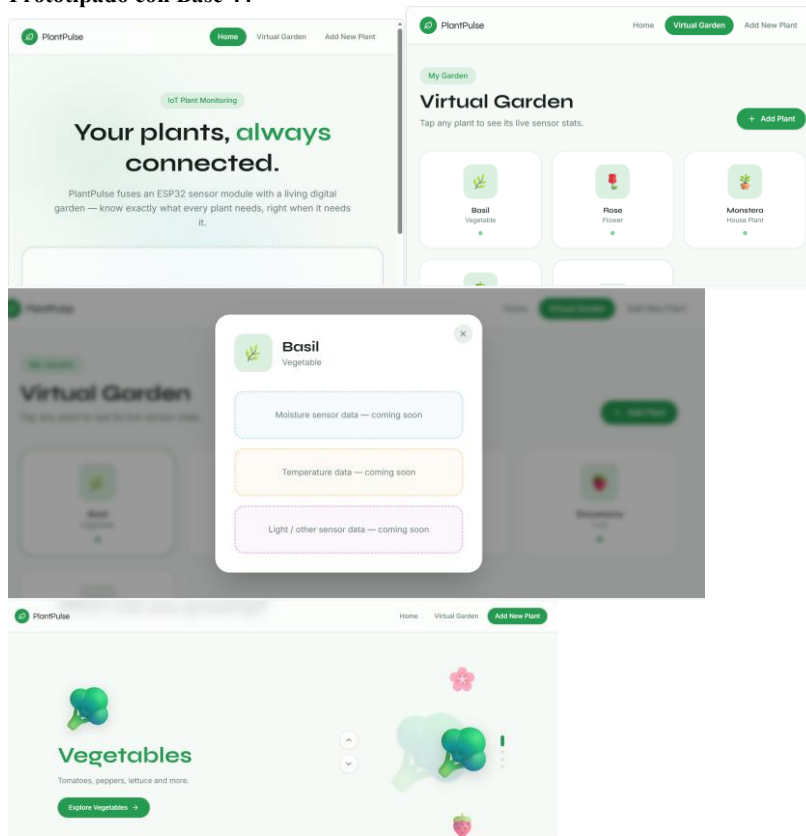
modelos de datos principales:

- **SensorReading:** Almacena el histórico de las lecturas de los sensores, registrando el `device_id`, el tipo de planta, el valor medido, la etiqueta de tiempo y el mensaje de estado.
- **WateringEvent:** Registra los eventos de actuación. Si el servidor recibe un comando manual de riego (`LED_ON`), se genera automáticamente un registro en la base de datos vinculando el ID del dispositivo, el tipo de planta y el origen del evento (manual o automático).

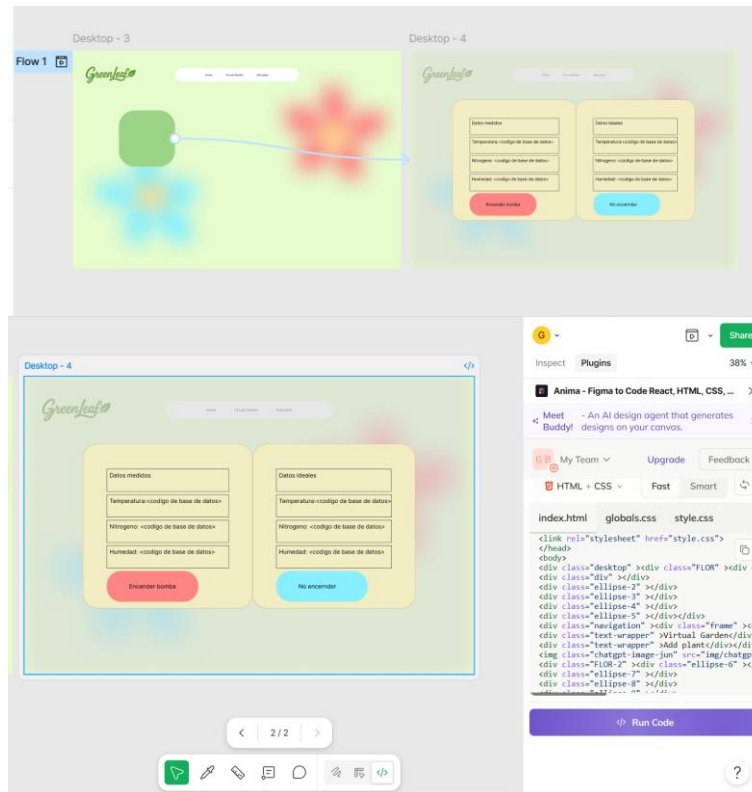
4.6 Interfaz de usuario

Commented [AV6]: Corregir y explicar imágenes

Prototipado con Base 44



Prototipado en figma



Para el desarrollo de la interfaz de usuario del proyecto se realizaron primero pruebas iniciales en Base44 con el objetivo de explorar ideas rápidas de estructura visual y definir cómo se presentaría la información dentro de la aplicación. Luego de esta etapa se decidió continuar el diseño en Figma, ya que ofrece mayor control sobre la distribución visual, los componentes y la organización de cada pantalla, además de facilitar el proceso de convertir el diseño a código posteriormente. Durante esta fase el enfoque principal se ha concentrado en la pantalla **Virtual Garden**, debido a que representa el núcleo de la aplicación. En esta sección el usuario podrá visualizar los datos monitoreados por los sensores en tiempo real, compararlos con los valores ideales registrados en la base de datos según el tipo de planta y, a partir de esa información, recibir la opción de activar o no el sistema de riego. De esta manera, la interfaz busca centralizar el monitoreo y la toma de decisiones dentro de un espacio visual claro e intuitivo.

Para convertir el diseño de Figma a programación web se contemplan distintos procesos. El primero consiste en utilizar la estructura diseñada como guía visual y recrearla manualmente en HTML y CSS dentro del entorno de desarrollo, lo que ofrece mayor control y permite adaptar el diseño a las necesidades del sistema y a la conexión con la base de datos. El segundo proceso consiste en utilizar herramientas y complementos de Figma que permiten exportar parte del diseño como código base

HTML y CSS, acelerando la construcción inicial de la interfaz. A partir de ese código base se realizan ajustes dentro del editor de desarrollo para organizar estilos, adaptar elementos interactivos y conectar la interfaz con la lógica del sistema, permitiendo que los datos enviados por el ESP32 y almacenados en la base de datos puedan mostrarse de manera dinámica dentro de la aplicación.

5. Conclusiones y próximos pasos

El desarrollo del proyecto ha permitido establecer una base funcional para un sistema IoT orientado al monitoreo y riego automatizado de plantas en entornos domésticos. Durante esta primera etapa se logró definir la arquitectura general del sistema, seleccionar las tecnologías de hardware y software adecuadas e implementar la comunicación entre el microcontrolador ESP32, el broker MQTT, el servidor backend y la base de datos. Estos avances confirman la viabilidad técnica del proyecto y permiten contar con una infraestructura capaz de transmitir información en tiempo real y mantener la comunicación entre los distintos componentes del sistema. De igual manera, el proceso de diseño de la interfaz permitió definir la estructura visual principal de la aplicación y establecer la pantalla Virtual Garden como el espacio central de monitoreo y control del usuario.

Como próximos pasos del proyecto se contempla finalizar el desarrollo del sitio web y llevar el diseño realizado en Figma a código funcional en HTML y CSS para su integración con el backend. También se continuará con las pruebas e integración del módulo físico de sensado, utilizando el ESP32, el relé, la bomba peristáltica y el sensor multiparámetro ya disponibles para validar el funcionamiento del sistema en condiciones reales. Finalmente, se trabajará en la visualización dinámica de los datos sensados dentro de la plataforma web, conectando la información enviada por el hardware con la base de datos y la interfaz de usuario, con el objetivo de unificar el sistema completo y lograr un monitoreo y control de riego automatizado desde una sola plataforma.

Referencias

- Acosta Gallo, E. D., & Cuaical Angulo, C. R. (2021). Desarrollo de un sistema automático de irrigación para huertos caseros por medio de IoT con software y hardware libre [Trabajo de titulación, Universidad Politécnica Salesiana]. <https://dspace.ups.edu.ec/handle/123456789/20382>
- Angulo Martínez, W. A. (2025). Desarrollo e implementación de un sistema de riego inteligente basado en internet de las cosas para cultivos protegidos en Aguascalientes, México [Tesis de maestría, Universidad Autónoma de Aguascalientes]. 483563.pdf
- Gómez Zea, J. M., Falcón Pérez, F., Jesús Magaña, J. Á., Carrera Belueta, C. M., González de la Cruz, R., Torres Hernández, J. J., & de la Cruz Alvarez, J. (2025). Sistema de riego por aspersión IoT para producción controlada de forraje verde hidropónico en traspatio, como alternativa para la autosuficiencia alimentaria. *Innovación y Desarrollo Tecnológico Revista Digital*, 17(3), 1397. https://iydt.wordpress.com/wp-content/uploads/2025/08/3_45_sistema-de-riego-por-aspersion-iot-para-produccion-controlada-de-forraje-verde-hidroponico-en-traspatio-como-alternativa-para-la-autosuficiencia-alimentaria.pdf
- Hasib, A., & Ahsanul Sarkar Akib, A. S. M. (2026). An IoT-based smart plant monitoring and irrigation system with real-time environmental sensing, automated alerts, and cloud analytics. arXiv. <https://arxiv.org/abs/2601.15830>
- Hernández González, A. G., Hernández Sánchez, D. L., García Vázquez, O. O., & Fernández

- Jiménez, J. L. (2025). Sistema de riego automatizado con Raspberry Pi, ESP32 y comunicación vía MQTT y HTTP. *Revista Ingeniantes*, 1(2), 61.
- Introduction of message queue telemetry transport protocol (MQTT)*. (2020, July 23). GeeksforGeeks. <https://www.geeksforgeeks.org/computer-networks/introduction-of-message-queue-telemetry-transport-protocol-mqtt/>
- lily-osp. (2025). smart-farming-esp32. GitHub. <https://github.com/lily-osp/smart-farming-esp32>
- Oppong, R. A. (2025). Integration of IoT-based, sprinklers, embedded systems, data and cloud computing for smart irrigation management. *World Journal of Advanced Research and Reviews*, 25(3), 126–151. <https://doi.org/10.30574/wjarr.2025.25.3.0695>
- Perea Alvarado, F. C., & Bustamante Briones, G. A. (2024). Prototipo de sistema de riego automatizado con control de humedad por histéresis y monitoreo IoT para la reducción del recurso hídrico [Trabajo de titulación, Universidad Politécnica Salesiana]. <https://dspace.ups.edu.ec/handle/123456789/30472>
- Prokosch, T. (2025, 17 de junio). MQTT protocol explained: The complete guide. *Cedalo*. <https://www.cedalo.com/blog/complete-mqtt-protocol-guide>
- ¿Qué es MongoDB? (s.f.). MongoDB. <https://www.mongodb.com/es/docs/manual/>
- Richard-Roman. (2026). RiegoJardin-ESP32. GitHub. <https://github.com/Richard-Roman/RiegoJardin-ESP32>
- Ruiz, F. Y. (2025). Smart irrigation system using IoT and LSTM for optimal water management [Tesis de maestría, University of Texas at Arlington]. MavMatrix. https://mavmatrix.uta.edu/electricaleng_theses/399
- WebSocket and its difference from HTTP*. (2019, 4 de diciembre). GeeksforGeeks. <https://www.geeksforgeeks.org/web-tech/what-is-web-socket-and-how-it-is-different-from-the-http/>